MLDL
PRACTICAL 7

Name: Payal Dara
Class: D15C
Roll no: 56
Batch: C

AIM:-

Build an Artificial Neural Network (ANN) using Keras/TensorFlow with hyperparameter tuning.

---

1. Dataset Source

Dataset Name: Banknote Authentication Dataset
Source: UCI Machine Learning Repository
Link:
https://raw.githubusercontent.com/jbrownlee/Datasets/master/banknote_authentication.csv

---

2. Dataset Description

The dataset is used to classify whether a banknote is authentic or fake based on numerical features.

Dataset Characteristics

- Instances: 1372
- Features: 4
- Target: class (0 or 1)

Features

- Variance
- Skewness
- Curtosis
- Entropy

Target

- 1 → Authentic
- 0 → Fake

---

3. Mathematical Formulation of ANN

y=f(∑wixi+b)y=f(∑wixi+b)

Sigmoid function:

σ(x)=11+e−xσ(x)=1+e−x1

Loss (Binary Cross-Entropy):

Loss=−[ylog(p)+(1−y)log(1−p)]Loss=−[ylog(p)+(1−y)log(1−p)]

---

4. Algorithm Limitations (ANN)

- Needs large data
- Computationally expensive
- Risk of overfitting
- Requires tuning
- Less interpretable

---

5. Methodology / Workflow

Dataset → Preprocessing → Train-Test Split
→ ANN Model → Training → Evaluation → Graph

---

6. Performance Analysis

Metrics:

- Accuracy
- Loss

Interpretation:

- High accuracy → good performance
- Low loss → better learning

---

7. Hyperparameter Tuning (ANN)

Parameters Tuned:

- Number of neurons
- Number of hidden layers
- Epochs
- Batch size

Effect:

- Improves model performance
- Helps avoid overfitting
- Provides better generalization

CODE:-

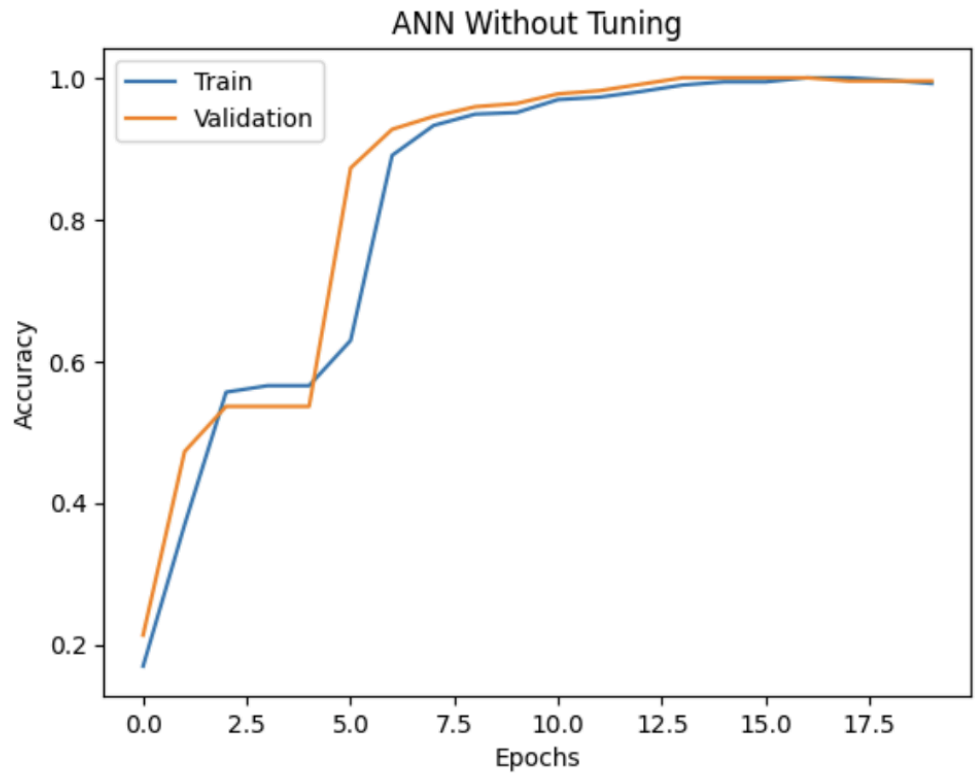
Without tuning

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense


# Load dataset

url =
"https://raw.githubusercontent.com/jbrownlee/Datasets/master/banknote_a
uthentication.csv"

df = pd.read_csv(url, header=None)

df.columns = ['variance', 'skewness', 'curtosis', 'entropy', 'class']


# Preprocessing

X = df.drop('class', axis=1)

y = df['class']


X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)


# ANN Model

model = Sequential()

model.add(Dense(8, activation='relu', input_dim=4))

model.add(Dense(4, activation='relu'))

model.add(Dense(1, activation='sigmoid'))


model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])


# Train

history1 = model.fit(X_train, y_train, epochs=20, batch_size=10,
validation_split=0.2)


# Evaluate

loss1, acc1 = model.evaluate(X_test, y_test)

print("WITHOUT TUNING Accuracy:", acc1)
```

```
# Graph
plt.figure()
plt.plot(history1.history['accuracy'], label='Train')
plt.plot(history1.history['val_accuracy'], label='Validation')
plt.title("ANN Without Tuning")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

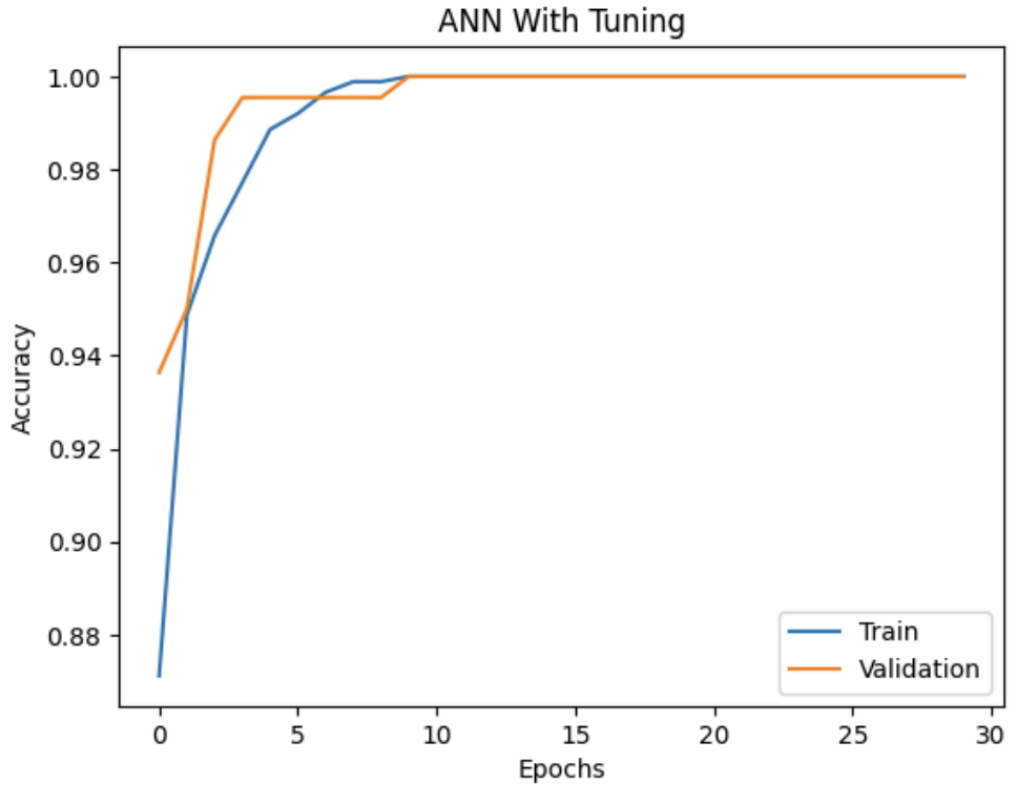
```
WITHOUT TUNING Accuracy: 0.9890909194946289
```



With tuning

```
# Tuned Model (changed neurons, epochs, batch size)

model2 = Sequential()
model2.add(Dense(16, activation='relu', input_dim=4))
model2.add(Dense(8, activation='relu'))
model2.add(Dense(1, activation='sigmoid'))

model2.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

# Train with different parameters
history2 = model2.fit(X_train, y_train, epochs=30, batch_size=5,
validation_split=0.2)

# Evaluate
loss2, acc2 = model2.evaluate(X_test, y_test)
print("WITH TUNING Accuracy:", acc2)

# Graph
plt.figure()
plt.plot(history2.history['accuracy'], label='Train')
plt.plot(history2.history['val_accuracy'], label='Validation')
plt.title("ANN With Tuning")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

```
WITH TUNING Accuracy: 1.0
```



Interpretation:-

The graphs show training and validation accuracy over epochs. The tuned model shows improved learning and better convergence. A smaller gap between training and validation accuracy indicates better generalization.

CONCLUSION:-

The graphs show training and validation accuracy over epochs. The tuned model shows improved learning and better convergence. A smaller gap between training and validation accuracy indicates better generalization.